TP - Shared Memory: Tiled Matrix Multiplication and Transposition

Oguz Kaya
oguz.kaya@universite-paris-saclay.fr

Part 1

The exercises: two programs, six kernels

Last week's lab ended on a dead end: with A stored by rows and B by columns, no mapping of one thread per element of C makes the reads of *both* operands coalesced – a warp of 32 elements of C spreads over columns (then B is strided) or over rows (then A is strided). This lab is about the way out, **shared memory**: a block loads data in the order the layout likes, and uses it in the order the computation likes. Two programs:

- **matmul-shmem.cu**: the matrix product $C = AB$ of TP3, with its five kernels and the cuBLAS benchmark *given* with their solutions, and a sixth kernel to write, which multiplies 32×32 tiles of A and B through shared memory;
- **transpose.cu**: the transposition A^T of an $N \times N$ matrix, five kernels to write – blocks of one thread, warps horizontal on A, warps vertical on A, through a padded shared tile, and in place.

Except the first transposition kernel, every kernel to write uses 2D blocks of 32×32 threads (the macro `BSXY` in both programs), which is also the side of the shared tiles; a warp is 32 consecutive values of `threadIdx.x` with the same `threadIdx.y`: one row of a block. N is any positive integer: the tiles of the last row and of the last column of blocks stick out of the matrices, and every kernel must round its grid up and guard the threads that fall outside.

Each program is a skeleton that compiles and runs from the start: its `main()` launches every kernel, checks the result against the CPU, and prints `ok` with the time and the performance (GFLOP/s for the product, GB/s moved for the transposition) – or `WRONG` until the kernel is written. For each kernel to write, write its body following the comment block above it, and set its `dim3` grid and block dimensions in `main()` (as given, it runs with 1 block of 1 thread). Answer the questions of each exercise in the comment block above the kernel, and put the timing tables (Ex. 1b and 6b) in a comment at the top of the file. The matrix size is the constant `n` at the top of `main()`: 250 for the product and 1000 for the transposition, both *not* multiples of 32 so that the guards are tested; change it there and rerun. Compile and run with `nvcc -O2 -arch=native matmul-shmem.cu -o matmul-shmem -lcublas` (and the same for `transpose.cu`, without `-lcublas`), or paste the file into `cuda.godbolt.org`, which runs it on a Tesla T4. `matmul-shmem.cu` calls cuBLAS, so it *must* be linked with `-lcublas`: on godbolt, the compiler options of the pane that runs the program must read `-O2 -lcublas` (`-O2` alone for `transpose.cu`); without `-lcublas` the program does not link. Both programs run there up to $N = 2048$ for the product and $N = 4096$ for the transposition.

Two rules from the lecture, which the check will not forgive you for forgetting:

- a thread that reads from a shared tile what *another thread* loaded must wait for it: `__syncthreads()` between the load and the use, and again before the tile is overwritten;
- *every* thread of the block must reach *every* `__syncthreads()`, including the threads that fall outside the matrix.

The pictures next to each part use $N = 14$ and 4×4 blocks and tiles; the code uses 32×32.

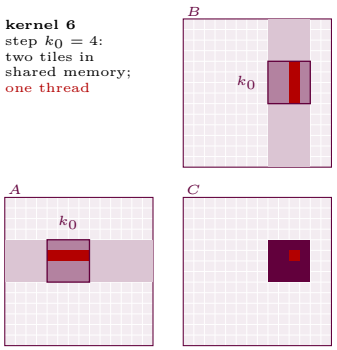
Part 2

Matrix multiplication through shared memory tiles

The layouts are those of TP3: A and C are stored by rows, B by columns, and $C(i,j) = \sum_k A(i,k)\,B(k,j)$. The blocks and the mapping are those of kernel 4: a block of 32×32 threads computes a 32×32 tile of C, one element per thread, with `threadIdx.x` running along the row of C. What changes is where the operands come from.

Instead of walking alone along row i of A and column j of B, the block walks along the *band* of 32 rows of A and the *band* of 32 columns of B that its tile of C needs, 32 columns of A and 32 rows of B at a time. At each step $k_0 = 0, 32, 64, \ldots$:

1. the block copies two 32×32 tiles into shared memory: the tile of A at its rows and columns $k_0 \ldots k_0 + 31$, and the tile of B at rows $k_0 \ldots k_0 + 31$ and its columns – one element of each tile per thread;
2. each thread multiplies its row of the A tile by its column of the B tile (an inner product of length 32) and adds the result to its element of C.



the block of the tile $C(4 \ldots 7, 8 \ldots 11)$ at step $k_0 = 4$: the two tiles it holds in shared memory, and one thread's row, column and element

After the last step, the tile of C is complete and stored. Every element of A and B is now read from global memory $N/32$ times instead of N times. Do not forget to put barriers in the appropriate places, to avoid data races among the different warps of the block.

Three things to get right:

- **Where does the partial sum of $C(i,j)$ live** while the block walks: in a shared tile of C, or in a register / variable of the thread? Who else needs it? Decide, and justify your choice in the comment block.
- **Both tile loads must be coalesced.** A warp is 32 consecutive `threadIdx.x`: at the load it must read 32 *consecutive* floats of A, and 32 *consecutive* floats of B. Look at the two layouts and decide which index of each tile `threadIdx.x` must run along. The element a thread loads need not be one it will multiply: the tile is shared, that is the point.
- **The shared tiles must be padded**, `[BSXY][BSXY + 1]`: with rows of 32 floats, a warp that reads or writes a column of a tile hits a single bank, 32 times. Which of your accesses to the tiles are rows, and which are columns?

Remember the guards: the tiles of the last row and column of blocks stick out of the matrices, and so does the last step k_0 of every block. A thread whose element is outside still takes part in the loads; think about what it should put in the tiles, so that the sums of the other threads stay right.

Ex. 1

a) Write `matmul_shmem` (kernel 6) and its launch dimensions, and check that it prints `ok` for $N = 250$.

b) Set `n` to 256, 512, 1024, 2048 (and 4096 on a lab GPU) in turn, rerun, and compare the GFLOP/s of kernel 6 with those of kernel 4 and of cuBLAS.

Part 3

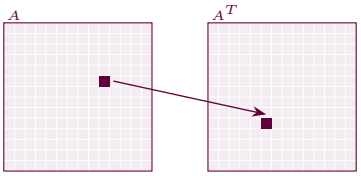
Matrix transposition

A is an $N \times N$ matrix stored by rows, $A(i,j)$ at `A[i * N + j]`, and A^T is stored by rows as well: $A^T(i,j) = A(j,i)$. A transposition does no arithmetic: its speed is the bandwidth it gets from global memory, and the program prints, for each kernel, the bytes moved (N^2 floats read and N^2 written) divided by the time, in GB/s. The skeleton contains a **copy** kernel, which copies A as it is (no transposition) into the output matrix, with blocks of 32×32 threads and `threadIdx.x` along the rows: a warp reads 32 consecutive floats and writes 32 consecutive floats, both coalesced. It moves the same bytes as a transposition with the best possible accesses: its bandwidth is the ceiling that the transpositions can approach, not pass.

Part 4

Blocks of one thread

A 2D grid of $N \times N$ blocks of a single thread, block (i,j) copying $A(i,j)$ to $A^T(j,i)$ – the kernel 1 of TP3, for a transposition.



kernel 1: block $(5, 9)$, one thread, copies $A(5, 9)$ to $A^T(9, 5)$

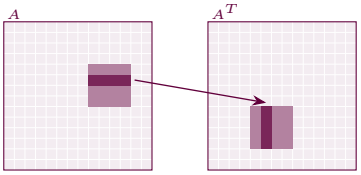
Ex. 2

a) Write `transpose_blocks` (kernel 1) and its launch dimensions.

Part 5

Warps horizontal on A

Block (b_x, b_y) of 32×32 threads handles the tile of A at rows $32b_y \ldots$ and columns $32b_x \ldots$: thread (t_x, t_y) copies one element of it to its place in A^T, with `threadIdx.x` running *along the row* of A: a warp touches A horizontally.



kernel 2: block $(2, 1)$, one thread per element; a warp reads a row of A, writes a column of A^T

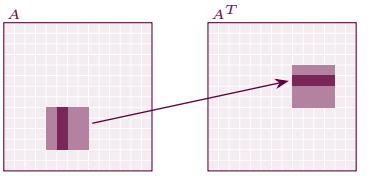
Ex. 3

a) Write `transpose_horizontal` (kernel 2) and its launch dimensions. Which of the two accesses of a warp is coalesced?

Part 6

Warps vertical on A

The same grid and the same tiles, with the thread indices swapped: `threadIdx.x` runs *down the column* of A, so that a warp touches A vertically (block (b_x, b_y) now handles the tile at rows $32b_x \dots$ and columns $32b_y \dots$).



kernel 3: block $(2, 1)$, one thread per element; a warp reads a column of A, writes a row of A^T

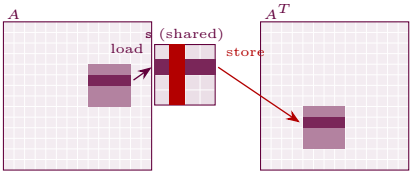
Ex. 4

a) Write `transpose_vertical` (kernel 3) and its launch dimensions. Which of the two accesses of a warp is coalesced now? Is there a mapping of one thread per element that coalesces both?

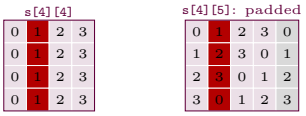
Part 7

Through a padded tile in shared memory

Block (b_x, b_y) handles again the tile of A at rows $32b_y \dots$ and columns $32b_x \dots$ (as in kernel 2), and moves it to its place in A^T – the tile at rows $32b_x \dots$ and columns $32b_y \dots$ of A^T – in two steps through a 32×32 tile in shared memory. At the *load*, thread (t_x, t_y) reads its element of the tile of A, `threadIdx.x` along the row, into the same position of the shared tile. At the *store*, it writes one element of the tile of A^T, again with `threadIdx.x` along the row, taking from the shared tile the value that belongs there. This time both A and A^T are touched *horizontally*, which keeps both global accesses coalesced since the matrices are stored by rows. How the shared tile is touched does not matter for coalescing – but make sure it is properly **padded**, `[BSXY][BSXY + 1]`, to prevent bank conflicts: see the picture below, with 4 banks instead of 32.



kernel 4: a warp loads a row of the tile of A into a row of `s`; a warp stores a row of A^T, read down a column of `s`



the padding, seen from the banks: in each word, its bank (4 banks in the picture, 32 on the GPU); grey: the unused padding

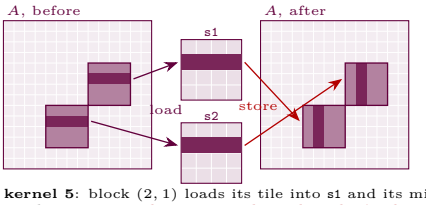
Ex. 5

a) Write `transpose_shmem` (kernel 4) and its launch dimensions. Which thread loaded the value a thread stores? In how many banks does a warp read the tile at the store, with and without the padding?

Part 8

In place, through shared memory

A is replaced by A^T, with no second matrix. The tile at block position (b_x, b_y) must end up, transposed, at position (b_y, b_x), and the tile at (b_y, b_x), transposed, at (b_x, b_y): a block that handles the two *mirror* tiles together can do it, since it holds both in shared memory before it overwrites either. Block (b_x, b_y) with $b_x > b_y$ loads its tile and its mirror into two padded shared tiles, then writes each one, transposed, at the other's place. The diagonal blocks ($b_x = b_y$) are their own mirror; the blocks below the diagonal ($b_x < b_y$) have nothing to do. `main()` gives the kernel a copy of A and checks it afterwards.



kernel 5: block $(2, 1)$ loads its tile into `s1` and its mirror into `s2`, then writes each, transposed, at the other's place: the dark row of a tile becomes a column (diagonal tiles: `s1` = `s2`)

Ex. 6

a) Write `transpose_inplace` (kernel 5) and its launch dimensions.

b) Set `n` to 256, 512, ..., 4096 in turn, rerun, and compare the bandwidth of the five kernels with the copy's.